PA2559- Software Metrics: GQM Project

1st Joshna Sindhuja Pitta
jopi23@student.bth.se
20030327-3023

2nd Naga Pujitha Pitta
napi23@student.bth.se
20020425-7885

3rd Dhanush Kumar Majji
dhma23@student.bth.se
20020214-8219

4th Tharun Kumar Marada
thma23@student.bth.se
2003126-3273

***Abstract*—This project examines the maintainability of Apache RocketMQ, a popular Java-based messaging platform, using object-oriented software metrics. Using the Goal-Question-Metric (GQM) technique, we examined five essential elements of the code—complexity, cohesion, coupling, size, and understandability—across five different RocketMQ versions. We collected and analyzed data using technologies such as CK Master, CodeMR, and Metrics reloaded . Our study highlighted modules that are more complex and difficult to maintain, providing developers with useful information into how to enhance code quality and structure in future updates.**

***Index Terms*—Software Maintainability, Object-Oriented Metrics, Goal-Question-Metric (GQM), Apache RocketMQ, Code Complexity.**

I. Introduction

Object-Oriented (OO) metrics have an importance for evaluating the internal quality of software built around object-oriented principles. They help developers figure out whether a system is maintainable, reusable, testable, and easy to understand. These metrics focus on important aspects of the code, such as how well a class's components work together (cohesion), how much different classes rely on one another (coupling), how complex the code is, how large it is, and how inheritance is used.

OO metrics are a useful tool for identifying potential design issues, highlighting parts of the code that may be difficult to maintain, and tracking how software quality changes over time. Compared to traditional procedural programming, object-oriented software arranges data and behavior into classes that communicate via methods. This makes it critical to find out how the system is put together. By evaluating these structural aspects, developers can ensure that the software is well-organized, modular, and loosely coupled—all of which are necessary for long-term maintenance and improvement.

The primary goal of this paper is to practically assess the maintainability of Apache RocketMQ, a Java-based cloud native messaging and streaming technology, using a Goal-Question-Metric (GQM) methodology. We hope to identify RocketMQ modules (Java packages) that are more difficult to maintain by systematically analyzing internal software attributes such as code structure (cohesion and coupling), complexity, understandability, and size across five major releases (5.3.1, 5.3.0, 5.2.0, 5.1.4, and 5.1.3). The findings of this research will help to allocate more experienced developers to modules that require the most attention and resources.

Section 2: Research Methodology - The study uses a long-term observational approach to investigate how RocketMQ's maintainability changes over time. It describes how data was collected and analyzed using techniques such as descriptive statistics, boxplots, and correlation analysis to identify patterns and trends in software maintainability.
Section 3: GQM Tree - Describes our hierarchical mapping from aim to questions and associated metrics, including justifications, descriptions, and scale types for each metric.
Section 4: Results – Provides detailed findings for each RocketMQ module, including size metrics, structural cohesion/coupling, complexity, and understandability trends.
Section 5: Discussion – Further interprets the findings, focusing on relationships between size, structure, and complexity. It also reflects on the project's actual learning experiences and the use of the GQM framework.
Section 6: Conclusions and future work summarize the main findings and discuss future directions.

II. Research Methodology

A. Selected Study Type: Case Study

For this project, we decided to do research using a case study. A case study allows us to thoroughly examine a single system in its real-world setting, which is ideal when attempting to understand complex situations without changing any of the variables. In our case, we're investigating the maintainability of Apache RocketMQ, an open-source, cloud-native messaging platform written in Java. A case study approach is appropriate because our focus is on analyzing various versions of the same software system (versions 5.1.3, 5.1.4, 5.2.0, 5.3.0, and 5.3.1). It allows us to explore RocketMQ's evolution over time and gain valuable insights into its maintainability.

Justification for Choosing a Case Study: We chose the case study method for several key reasons. First, RocketMQ is a complex, real-world system that requires in-depth, context-specific analysis, particularly when looking at its structure at the Java package level. A case study approach allows us to focus specifically on this system and gain a

thorough understanding of its maintainability. Unlike broader methods such as experiments or surveys, which may overlook system-specific details, the case study allows us to capture RocketMQ's distinct architectural and modular characteristics that influence its maintainability.

Second, the experiment is longitudinal, examining five versions of RocketMQ across time. Case studies are particularly well-suited to longitudinal designs since they allow for the tracking of changes, trends, and technical debt evolution over multiple releases.

Third, the case study is consistent with the goal-question-metric (GQM) framework used in this project. The GQM approach benefits from a rigorous, goal-driven analysis of system modules, which is naturally supplemented by case studies. Furthermore, case studies are frequently utilized and validated in software engineering research to assess internal quality features such as cohesion, coupling, complexity, and size.

B. Tools for Data Collection

To collect metrics for our empirical study on Apache RocketMQ maintainability, we first used MetricsReloaded and SonarQube, both of which are Java-compatible and provide useful insights into object-oriented (OO) metrics. However, we discovered that some of the critical metrics needed for our analysis were either missing or incomplete. To close this gap, we chose to supplement our analysis with CK Master and CodeMR, which provided more comprehensive metric extraction. By combining these tools, we were able to collect a broader set of metrics that corresponded to the internal quality attributes outlined in our GQM tree.

CK Master: We chose CK Master as one of our tools because it is a small command-line utility that analyzes compiled Java bytecode (.class files) and generates important object-oriented metrics at the class level. It was particularly useful for our research because it includes several important maintainability metrics, such as LCOM (Lack of Cohesion in Methods), CBO (Coupling Between Object Classes), RFC (Response for a Class), WMC (Weighted Methods per Class), DIT, and Lines of Code (LOC). These metrics are commonly used in software quality research and have been examined by numerous empirical studies. Another reason we liked CK Master is that it outputs data in a well-structured CSV format, making it simple to process and analyze the results further.

To generate metrics with CK Master, we first compiled the RocketMQ project with Maven, as CK Master requires compiled '.class' files. Following compilation, the tool analyzed each class to extract various object-oriented metrics such as Lines of Code (LOC), Weighted Methods per Class (WMC), Response for a Class (RFC), Coupling Between Object Classes (CBO), Lack of Cohesion in Methods (LCOM), TCC and Depth of Inheritance Tree (DIT). These metrics were saved in a structured CSV file, providing detailed insights at the class level.

Because CK Master only provides metrics at the class level, we had to aggregate this information to the package level for our maintainability analysis. We automated this process with Python, converting class-level metrics into package-level metrics. This made it easier to compare maintainability trends across different RocketMQ versions. We also used spreadsheets to validate and organize the final results.

Code MR: For additional metric collection, we used CodeMR, which integrates with IntelliJ IDEA and offers a user-friendly interface for exploring object-oriented metrics. It was especially useful in identifying the remaining metrics required to meet the requirements of our GQM tree, such as Number of Classes (NOC), Number of Methods (NOM), and Average Length of Method. CodeMR enabled us to export the metric data as CSV files, making it easier to integrate with the rest of our data. This allowed us to fill in the gaps left by other tools and ensure that we had a complete set of metrics for analyzing RocketMQ's maintainability [2].

To use CodeMR, we first downloaded and installed the CodeMR plugin in IntelliJ IDEA. When the RocketMQ project was loaded into the IDE, the plugin automatically analyzed the source code and produced a number of object-oriented metrics. These metrics could be viewed through CodeMR's built-in visual dashboards, but we chose to export the data as CSV files for ease of processing. Like CK Master, CodeMR provided class-level metrics, so we used Python scripts to aggregate the data to the package level. This enabled us to align the extracted metrics—such as the Number of Classes (NOC), Number of Methods (NOM), and Average Length of Method—with our GQM tree and conduct a more structured analysis of RocketMQ's maintainability across different versions.

While we were able to collect most of the object-oriented metrics using CK Master and SonarQube, we ran into an issue when attempting to calculate the comment density metric. This metric requires the number of comment lines (CLOC) about the total lines of code (LOC), calculated as 'CLOC / LOC'. Unfortunately, CK Master did not offer CLOC directly. However, because we had previously used MetricsReloaded [1], which includes CLOC in its output, we were able to extract the desired values from it. We successfully computed the comment density metric by dividing cloc/loc.

C. Data Analysis

To evaluate and show the metrics data obtained from Apache RocketMQ, we used descriptive statistics, visualization tools, and correlation analysis. These methodologies were chosen to provide statistical and visual insights into the maintainability

of RocketMQ modules across five releases.

First, we used descriptive statistics to summarise the distribution and central tendency of each indicator at the package (module) scale. This involved determining the mean, median, maximum, minimum, and standard deviation of metrics, including LCOM, CBO, RFC, WMC, LOC, and comment density. These statistics offered a clear insight into the overall structural characteristics of RocketMQ modules, assisting in the identification of any outliers or abnormal patterns in the code.

Secondly, we applied a variety of visualization techniques to present the data in an accessible and interpretable manner. Boxplots were utilized to show the spread and distribution of metrics across modules, allowing for easier detection of extreme values and comparison of distributions. Bar charts were used to compare module sizes and understandability parameters like comment density and average method length.

To explore correlations between different internal features, we used Pearson correlation coefficients. This allowed us to determine whether and how measures such as size (LOC) were connected to complexity and whether coupling (CBO) influenced cohesion (LCOM). The correlation results supported or challenged intuitive notions, such as larger modules being more complex or difficult to maintain.

III. GQM TREE

A. Formulating Relevant and Answerable Questions

To evaluate the maintainability of Apache RocketMQ, we set the following goal: "Analyze the internal code properties of RocketMQ modules to identify those that may be difficult to maintain." According to the project description, we concentrated on five key internal characteristics of object-oriented software systems: code structure (especially cohesion and coupling), code complexity, code understandability, and code size. These characteristics guided us in developing a clear and measurable question within the GQM (Goal-Question-Metric) framework, allowing us to stay focused on the project's objectives while conducting an organized evaluation of RocketMQ's maintainability.

1. Code Structure – Cohesion:
Q1: How cohesive are the methods within a RocketMQ module's classes?
This question targets cohesion, as cohesive classes are easier to maintain and understand.

2. Code Structure – Coupling:
Q2: How coupled is a RocketMQ module with other modules?
This question focuses on coupling, which affects modularity and changeability.

3. Code Complexity:
Q3: How complex is the logic within a RocketMQ module?
This question concerns complexity, which directly impacts understanding and testing effort.

4. Code Understandability:
Q4: How understandable is a RocketMQ module's code?
Understandability is crucial to minimize start-up and maintenance work.

5. Code Size:
Q5: What is the size of a RocketMQ module in terms of code and structure?
This final question covers size-related factors, which often correlate with maintainability challenges.

B. *Selecting and Describing Metrics with Justifications*

Two relevant OO metrics support each question in the GQM tree to provide multiple perspectives on the attribute being assessed.

Q1: How cohesive are the methods within a RocketMQ module's classes?
Metrics: LCOM, TCC
Goal: Evaluate the classes' internal cohesiveness and functional focus.

Metric 1: LCOM (lack of cohesion in methods)
Why this metric fits the question: By determining whether the methods in a class use common instance variables, LCOM is intended to assess how well the methods link to each other. A high LCOM suggests that a class is attempting to execute too many unrelated tasks, which is against the single-responsibility principle in the case of RocketMQ, which includes classes that oversee intricate messaging processes [6].

Reason for selection: Because developers must comprehend a large number of interconnected functionalities, classes with high LCOM are frequently more difficult to maintain. This statistic helps to locate classes that can benefit from reworking.

Scale Type: Absolute
Explanation: Scale Type defines how the metric values are interpreted or measured. Absolute scale means the metric is expressed as a raw count or value without normalization or relative comparison.

Metric 2: TCC (Tight Class Cohesion)
Why this metric fits the question: Compared to the total number of available method pairs, TCC counts the number of directly related method pairs, those that access shared attributes. It presents a constructive view of the connection.

Reason for selection: Detailed justification for selection: TCC has strength in class design, as compared to LCOM, which

draws attention to issues. TCC assists in determining which classes in RocketMQ are well-structured and operationally cohesive, as modules should ideally have tiny, narrowly focused classes (such as producers, consumers, and brokers).

Scale Type: Ratio
Explanation: TCC is a ratio ranging from 0 to 1, representing the proportion of cohesive method pairs. Arithmetic operations (such as division) have a purpose, and it has an absolute zero. As a result, a ratio scale is used to measure it.

Q2: How coupled is a RocketMQ module with other modules?
Metrics: CBO, RFC
Goal: Assess inter-module dependency and behavioral complexity.

Metric 1: CBO (Coupling Between Object Classes)
Why this metric fits the question: The number of classes that a certain class is connected to is counted by CBO. This includes any classes with which it communicates via affiliations, inheritance, or method calls [3].

Reason for selection: Because modifications to one module might affect other modules, excessive coupling in RocketMQ limits flexibility and raises maintenance expenses. This interdependence is explicitly quantified by CBO, which also assists in identifying classes that are firmly tied to external logic and ought to be abstracted or detached.

Scale Type: Absolute
Explanation: CBO keeps track of how many different classes a class is associated with. Because it only counts distinct items and does not provide operations like averaging in the strictest statistical sense (although averages are frequently used in software metrics), this count-based metric is usually regarded as an absolute scale.

Metric 2: RFC (Response for a Class)
Why this metric fits the question: RFC calculates the number of methods that could be used to reply to a message given to a class. Both local and class-based methods are taken into consideration [7].

Reason for selection: The class with a high RFC is believed to have a broad interface or behavioral surface, making it more difficult to test, comprehend, and reuse. Such classes (such as client or network handlers) are frequently crucial in RocketMQ, and it is easier to prioritize them for test coverage and documentation when they are identified through RFC

Scale Type: Absolute
Explanation: RFC is a count of possible operation methods. Similarly to CBO, it is an absolute metric that shows how many distinct behaviors an object might cause. It does not have a natural zero for the interpretation of ratios other than counting.

Q3: How complex is the logic within a RocketMQ module?
Metrics: Depth of Inheritance Tree (DIT), WMC
Goal: Measure logical complexity and cognitive load.

Metric 1: Depth of Inheritance Tree (DIT)
Why this metric fits the question:DIT counts the number of ancestor classes that a specific class inherits from. In object-oriented systems such as RocketMQ, which runs on Java, inheritance is a common way to reuse and organize code. A higher DIT value indicates that a class is deeper in the inheritance hierarchy, making its logic more difficult to understand, as a developer must trace through more layers of inherited behavior [4].

Reason for selection: RocketMQ modules frequently deal with a variety of issues such as message queuing and I/O handling. When such modules have complex inheritance hierarchies, understanding a class's full behavior necessitates knowledge of all of its parent classes. This increases cognitive load and the risk of misinterpretation or bugs. As a result, DIT is an appropriate metric for capturing the logical complexity that comes from inheritance depth.

Scale Type: Absolute
Explanation: Absolute scale means the metric is expressed as a direct numeric value representing the exact measurement without normalization. For DIT, this means the metric is the integer count of levels in the inheritance hierarchy. There's no conversion to ratios, percentages, or categories, just the actual depth number

Metric 2: Weighted Methods by Class (WMC)
Why this metric fits the question: The complexity (often CC) of every method in a class is added up by WMC. It represents the entire logical weight of comprehending and upholding the class [4].

Reason for selection: A high WMC class indicates a high cognitive load, regardless of whether it uses a large number of difficult procedures or a small number of extremely complex ones. WMC assists in identifying complicated regions that require improved modularization in RocketMQ's key components, such as MessageStore or RemotingServer.

Scale Type: Absolute
Explanation: WMC adds up the intricacies of every method of a class. Since it does not accurately represent proportional relationships between values in the most rigorous statistical sense, this weighted value count is classified as an absolute scale [8].

Q4: How understandable is a RocketMQ module's code?
Metrics: Comment Density, Average Method Length

Goal: Assess how easily the code can be read and understood.

Metric 1: Comment Density
Why this metric fits the question: It determines the proportion of comment lines to all code lines.

Reason for selection: Well-commented code makes things easier to understand, especially for new team members or outside contributors. For cooperation in an open-source project like RocketMQ, this is essential. Important modules with a low comment density could require more thorough documentation.

Scale Type: Ratio
Explanation: This is the proportion of comment lines to total lines of code. It ranges from 0 to 1 (or more in certain circumstances) and allows for proportional interpretation. Thus, it is a ratio scale [8].

Metric 2: Average Method Length
Why this metric fits the question: It shows how many lines of code a class or module typically has for each method.

Reason for selection: Because long procedures may execute several tasks, they are frequently more difficult to comprehend and verify. Methods that are brief and targeted improve readability and organization. This measure aids in locating excessive RocketMQ methods that could be divided or streamlined.

Scale Type: Ratio
Explanation: This metric computes the average number of lines per method. It qualifies as a ratio scale since it includes a meaningful zero and allows for proportionate comparisons.

Q5: What is the size of a RocketMQ module in terms of code and structure?
Metrics: LOC, Number of Classes and Methods
Goal: Quantify the structural and volumetric size of each module.

Metric 1: Lines of Code (LOC)
Why this metric fits the question: LOC acts as a stand-in for complexity, effort, and possible technical debt and is a direct indicator of code size [5].

Reason for selection: While larger modules could have more features, they may also be a sign of low cohesiveness and complexity. Tracking LOC in RocketMQ makes it easier to see trends in growth, keep an eye on refactoring, and determine whether the codebase is getting too big to handle efficiently.

Scale Type: Absolute
Explanation: LOC is a simple count of a module's lines of code. It is an absolute metric since it counts occurrences rather than indicating meaningful ratios based on volume.

Metric 2: Number of Classes and Methods
Why this metric fits the question: These determine how many classes and methods make up a module's structural elements.

Reason for selection: A higher number could mean more functionality but also more work for testing and maintenance. Modular design principles may be broken if numerous classes or methods are consolidated into a single module. This measure in RocketMQ illustrates how duties are distributed throughout the system.

Scale Type: Absolute
Explanation: This is a basic count of how many classes and methods a module contains. It is an absolute scale metric since it is a straight count of discrete items, with no proportional or interval interpretation.

TABLE I
METRICS CATEGORIZATION BASED ON GQM QUESTIONS

Metric	Question	Categorization
Lack of Cohesion in Methods (LCOM)	Q1	Cohesion
Tight Class Cohesion (TCC)	Q1	Cohesion
Coupling Between Object Classes (CBO)	Q2	Coupling
Response For a Class (RFC)	Q2	Coupling
Depth of Inheritance Tree (DIT)	Q3	Complexity
Weighted Methods per Class (WMC)	Q3	Class Complexity
Comment Density	Q4	Understandability
Average Method Length	Q4	Understandability
Lines of Code (LOC)	Q5	Size
Number of Classes and Methods	Q5	Structural Size

TABLE II
SELECTED METRICS AND THEIR SCALE TYPES

Metric	Scale Type
Lack of Cohesion in Methods (LCOM)	Absolute
Tight Class Cohesion (TCC)	Ratio
Coupling Between Object Classes (CBO)	Absolute
Response For a Class (RFC)	Absolute
Depth inheritance tree(DIT)	Absolute
Weighted Methods per Class (WMC)	Absolute
Comment Density	Ratio
Average Method Length	Ratio
Lines of Code (LOC)	Absolute
Number of Classes and Methods	Absolute

IV. RESULTS

A. Overview of Apache RocketMQ

To better understand the overall structure of RocketMQ version 5.3.1, we examined the size of each of its 18 modules using two key metrics. Lines of Code (LOC) and Number of Classes (NOC). These two measures are critical for estimating the internal complexity of software modules. LOC tells us how much code is written in a module, whereas NOC shows how the code is arranged among classes. Together, they provide a solid foundation for detecting possible areas of complexity or risk. The data is provided in the table below.

TABLE III
NUMBER OF CLASSES (NOC) AND LINES OF CODE (LOC) PER ROCKETMQ MODULE

Module	NOC	LOC
rocketmq-acl	27	4591
rocketmq-auth	61	4450
rocketmq-broker	153	32031
rocketmq-client	149	30120
rocketmq-common	196	13064
rocketmq-container	12	1613
rocketmq-controller	68	4734
rocketmq-example	69	3850
rocketmq-filter	33	5168
rocketmq-namesrv	40	4169
rocketmq-openmessaging	15	1717
rocketmq-proxy	143	16144
rocketmq-remoting	269	18895
rocketmq-srvutil	4	336
rocketmq-store	170	35814
rocketmq-test	35	11662
rocketmq-tieredstore	36	6873
rocketmq-tools	74	12776

1) **Descriptive Statistics:** The results in the table below show that the average amount of lines of code per module is around 11,535, and the average number of classes is around 93.83. These data provide a baseline, but a deeper look at the standard deviation reveals that the size range is quite large. The standard deviation for LOC is 9,282.34, while for NOC it is 73.46, indicating significant variability in module size, as some are very small, while others are exceedingly large. This type of change is significant since larger, more complex modules are often more difficult to maintain, understand, and test.

TABLE IV
DESCRIPTIVE STATISTICS FOR NOC AND LOC

Metric	NOC	LOC
Count	18	18
Mean	93.83	11535.17
Std Dev	73.46	9282.34
Minimum	4	336
Maximum	269	35814

The smallest module, 'rocketmq-srvutil', has only 336 lines of code and 4 classes. It most likely serves a small, utility-based purpose with limited scope. In contrast, the largest module, 'rocketmq-store', includes 35,814 lines of code and 170 classes, making it more than 100 times larger than ' srvutil' in terms of LOC. This significant discrepancy shows that ' store' is one of the system's most important and potentially most difficult modules. Similarly, 'rocketmq-broker' (32,031 LOC, 153 classes) and 'rocketmq-client' (30,120 LOC, 149 classes) are both large and are likely to handle critical communication and messaging functions. These modules stand out as much larger than the others and need further research.

The range of data demonstrates this disparity: LOC runs from 336 to 35,814, while NOC ranges from 4 to 269. The median LOC is 6,020.5, significantly lower than the mean. This indicates that the majority of modules are smaller than the average, with a few really large modules skewing the total mean upward. This skewed distribution is common in large systems, as core modules expand over time as new features are added or responsibilities increase. Such growth is not always harmful, but it raises the danger of technical debt and maintainability issues.

2) **Visualizations:** To better understand the data, we used three different forms of visualizations: box plots, histograms, and line graphs. Each of these provides us with a unique perspective on the values and assists us in identifying significant trends.

box plots: The box plot allows us to quickly see how the numbers are distributed. It demonstrates what is considered normal and what is much outside the average range. This makes it easy to identify anything that jumps out too much, such as extremely high or low readings.

Histograms: The histogram displays how frequently specific ranges of values appear. It groups the numbers and shows us where the majority of the data is concentrated. This offers us a clear image of whether the numbers are generally small, huge, or distributed across a range.

line plot: The line plot connects the data one by one, allowing us to compare them side by side. It allows us to identify which numbers are higher or lower than others simply. This is useful for determining patterns or differences between objects.

Box Plot for LOC (Lines of Code): This boxplot in Fig. 1 shows the distribution of lines of code across all RocketMQ modules. Most modules are within an acceptable size range, but a few, such as 'rocketmq-store' and 'rocketmq-broker', are obvious outliers with excessively high LOC. These larger modules stand out as potential areas that may be more challenging to manage due to their size. The box plot demonstrates this imbalance, as the majority of modules are smaller and more densely packed below the median.

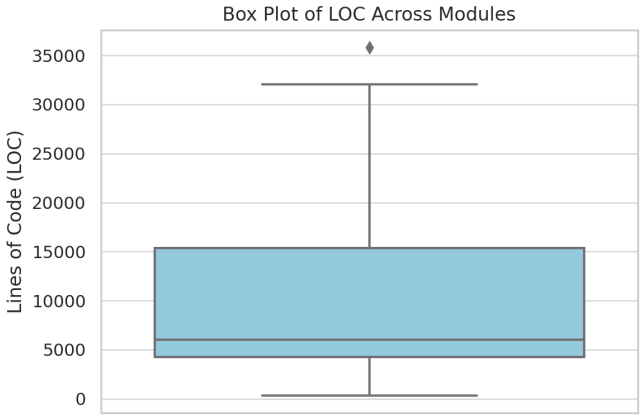


Fig. 1. Box Plot for LOC

Box Plot for NOC (Number of Classes) The box plot for the number of classes in Fig. 2 indicates a broad distribution among modules. While many modules have between 30 and 150 classes, some, such as 'rocketmq-remoting', go much beyond this limit. These are referred to as structural outliers, implying that certain areas of the system are more complicated in terms of organization. Modules with few or many classes may be too simple or too densely packed with logic.

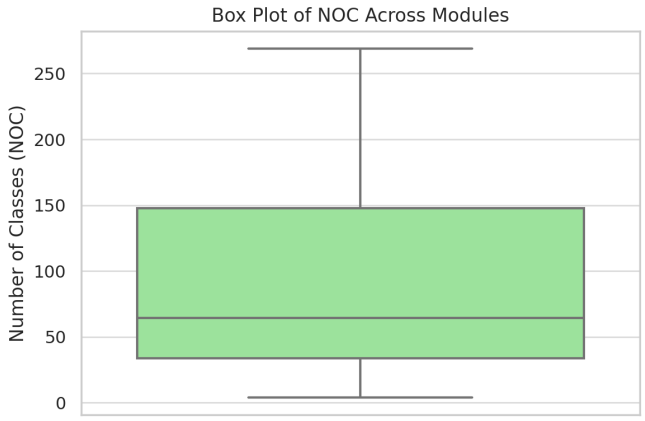


Fig. 2. Box Plot for NOC

Histogram for LOC: The LOC histogram in Fig. 3 shows us how module sizes are dispersed. Most RocketMQ modules are less than 15,000 LOC, with the majority falling between 3,000 and 10,000 LOC. However, there is a long tail on the right side, with a few modules exceeding 30,000 lines. This "right-skewed" structure indicates that only a small number of modules account for a considerable portion of the total codebase, potentially affecting scalability and maintainability.

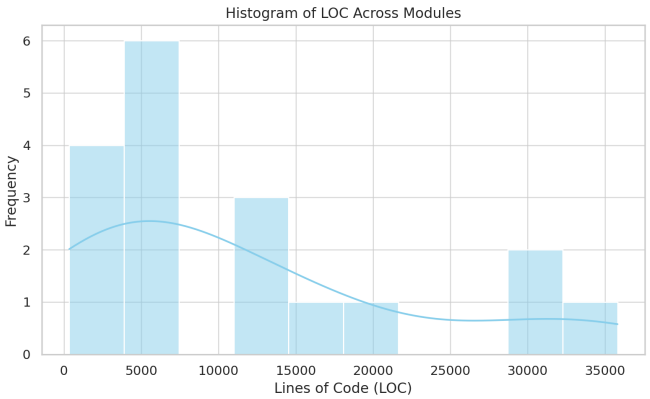


Fig. 3. Histogram for LOC

Histogram for NOC: The NOC histogram in Fig. 4 provides a similar story. Most modules have fewer than 100 courses, with many falling between 30 and 70. Nonetheless, a few outliers contain more than 200 classes, indicating a potential overload of functionality in a single module. This could be an indication of poor cohesion or a chance for refactoring. The histogram reveals that, while most modules are balanced, some have expanded significantly in terms of structure.

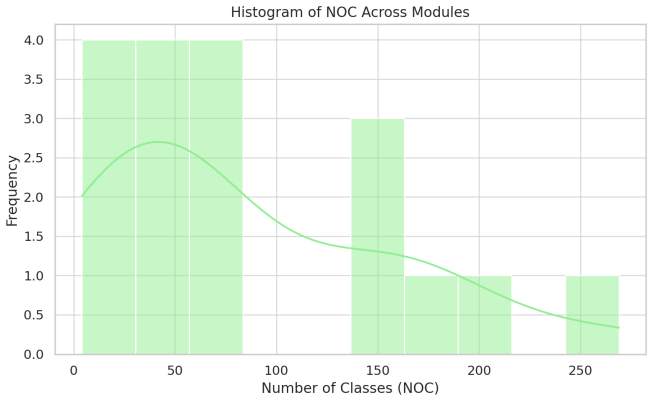


Fig. 4. Histogram for NOC

Line Plot for LOC: The line plot of LOC in Fig. 5 compares the code sizes of each module side by side. We can instantly see increases for modules such as 'rocketmq-store', 'broker', and 'client', indicating that they are the largest. 'srvutil' and 'container' are low-level modules with little code. This plot is handy for quickly comparing modules and identifying the top factors contributing to system complexity.

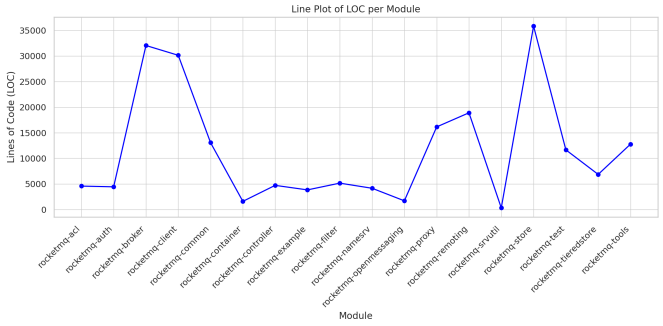


Fig. 5. Line plot for LOC

Line Plot for NOC: This line plot in Fig. 6 shows how the number of classes changes from one module to another. It follows the same pattern as LOC: modules with the most lines of code tend to have the most classes. 'Remoting', 'store', and 'broker' are also popular terms here. This emphasizes the idea that size and structure are inseparable, and the largest modules may also be the most difficult to handle.

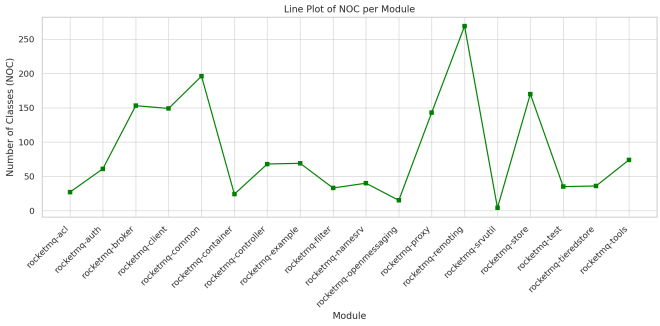


Fig. 6. Line plot for NOC

B. Evolution of RocketMQ

To analyze how RocketMQ's codebase size has evolved, we looked at the given five versions: 5.1.3, 5.1.4, 5.2.0, 5.3.0, and 5.3.1. In this evaluation, we used two primary metrics: lines of code (LOC) and number of classes (NOC). These metrics allow us to understand how the software has evolved, both in terms of raw size and structural complexity. The data was visualized using bar charts, and the trends demonstrate the project's expansion and modular development strategy.

TABLE V
ROCKETMQ VERSIONS EVOLUTION

Version	LOC	NOC
5.3.1	208007	1554
5.3.0	202529	1542
5.2.0	193267	1452
5.1.4	185852	1408
5.1.3	180416	1379

Lines of Code (LOC): The LOC metric shows a clear and consistent increase from version 5.1.3 to 5.3.1. Version 5.1.3 began with 180,416 lines, which increased gradually with each release until reaching 208,007 lines in version 5.3.1. This represents a total growth of approximately 27,591 lines of code. The most significant jumps occurred between versions 5.1.4 and 5.2.0, as well as between 5.2.0 and 5.3.0, indicating major development phases. This steady increase indicates that new features and functionalities were added, and the system was actively improved. However, a larger codebase increases the maintenance burden, so code organization and readability become more important as the project grows.

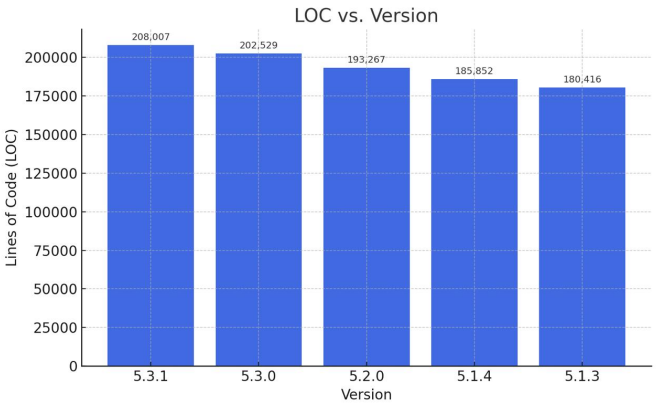


Fig. 7. LOC

Number of Classes: The NOC metric also shows an upward trend, rising from 1,379 classes in version 5.1.3 to 1,554 classes in version 5.3.1. The addition of 175 classes indicates that the RocketMQ project is not only growing in size but also becoming more modular. Adding more classes typically indicates a desire to break down complex functionalities into smaller, more manageable pieces. This can improve code reuse, maintainability, and understanding, particularly in large-scale systems such as RocketMQ. The increase in NOC is consistent with good object-oriented design practices, as the system evolves to handle new features and responsibilities.

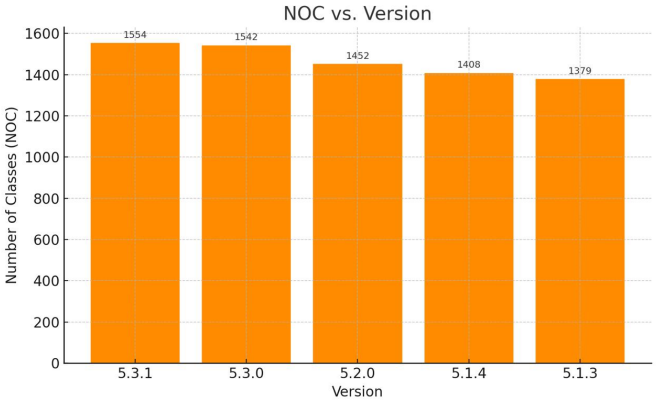


Fig. 8. LOC

Conclusion: In conclusion, RocketMQ has experienced consistent growth in both lines of code and the number of classes across the analyzed versions. The growing LOC implies ongoing development and integration of new capabilities. Simultaneously, the increase in NOC reflects a modularization strategy to help manage the increasing complexity. These trends suggest that RocketMQ is evolving not only in size but also in structure, indicating a well-managed and scalable project. Monitoring these metrics provides valuable information about the software's health and maintainability as it evolves.

C. GQM Answers:

Q1. How cohesive are the methods within a RocketMQ module's classes?

We utilized two well-known software quality metrics, LCOM (Lack of Cohesion in Methods) and TCC (Tight Class Cohesion), to assess how effectively the methods within each RocketMQ module cooperate. These metrics aid in our comprehension of the code's internal organization. To put it simply, LCOM indicates the degree of disconnection between methods within a class. Poor design is suggested if the LCOM score is high, which indicates that many processes do not communicate with one another or share data. Conversely, TCC assesses the degree of connectivity between a class's methods. A high TCC number indicates that the methods are collaborating closely and sharing data, which typically results in code that is easier to read and maintain.

LCOM(Lack of Cohesion in Methods) Analysis: We discovered that numerous RocketMQ modules had very high LCOM values, such as 'rocketmq-store' (mean 88,430), 'rocketmq-common' (66,823), 'rocketmq-client' (27,090) and 'rocketmq-proxy' (47,756). These figures are quite large, implying that the classes inside these modules are probably doing too many diverse tasks without much coordination. This type of design frequently produces code that is difficult to read, alter, or test. In contrast, modules like 'rocketmq-srvutil', 'rocketmq-example', and 'rocketmq-container' have very low LCOM values some even below 100, indicating that their methods are tightly connected and the classes are compact and concentrated. These are indicators of good cohesion.

LCOM						
module	5.3.0	5.3.1	5.2.0	5.1.3	5.1.4	Mean
rocketmq-acl	1638	1639	1638	1602	1638	1631
rocketmq-auth	2082	2082				2082
rocketmq-broker	18006	18674	16830	15968	16785	17252.6
rocketmq-client	27891	28942	27117	24706	26797	27090.6
rocketmq-common	69733	72154	69020	59092	64118	66823.4
rocketmq-container	627	627	627	561	561	600.6
rocketmq-controller	941	941	941	595	595	802.6
rocketmq-example	228	229	228	228	228	228.2
rocketmq-filter	7075	7075	7075	7075	7075	7075
rocketmq-namesrv	376	393	376	370	373	377.6
rocketmq-openmessaging	994	994	994	994	994	994
rocketmq-proxy	51074	51155	47460	42282	46812	47756.6
rocketmq-remoting	12434	12505	12081	10706	10706	11686.4
rocketmq-srvutil	11	11	11	11	11	11
rocketmq-store	91965	96032	91303	79590	83260	88430
rocketmq-test	3130	3130	3130	3106	3130	3125.2
rocketmq-tieredstore	4449	4295	4046	3832	3932	4110.8
rocketmq-tools	8019	8630	6495	6595	6608	7269.4

Fig. 9. LCOM metric values

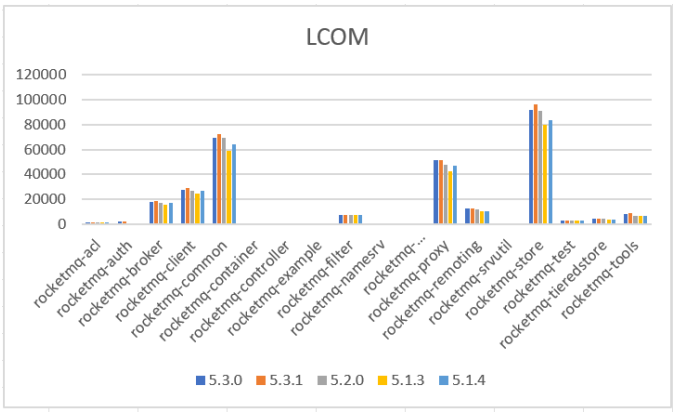


Fig. 10. LCOM graph

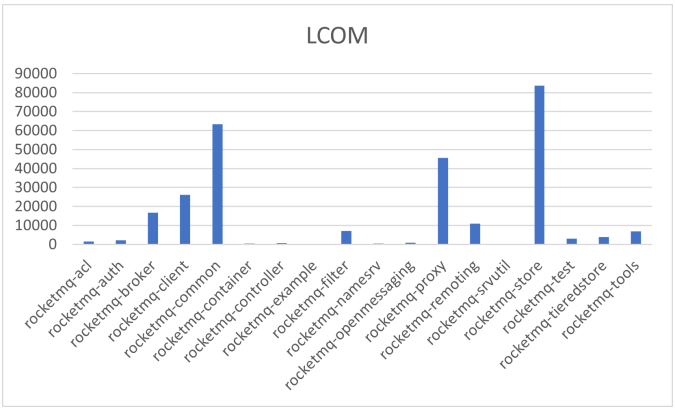


Fig. 11. LCOM Mean for each module

TCC Analysis: When we look at the TCC results, the picture becomes more clear. TCC provides a percentage-like statistic indicating how much of the class is working together. Modules with high TCC values, such as ”rocketmq remote” (mean = 89.32), ”rocketmq-proxy” (mean = 77.16) and ”rocketmq broker” (mean = 75.76), indicate that their techniques are well connected. Even with high LCOM levels, this is a positive indicator of robust cohesion. This indicates that some classes in those modules may be large and complex (thus the high LCOM), but are internally well-structured and cohesive. Modules such as ’rocketmq-container’, ’rocketmq-example’, and ’rocketmq-srvutil’, on the other hand, have both low LCOM and low TCC, which is typical of small, simple modules with fewer methods and simpler logic.

TCC						
module	5.3.0	5.3.1	5.2.0	5.1.3	5.1.4	Mean
rocketmq-acl	8.51069	9.986881	7.51069	6.724976	7.51069	8.048786
rocketmq-auth	24.02234	26.02234				25.02234
rocketmq-broker	76.98303	88.00679	73.43334	69.34408	71.0439	75.76223
rocketmq-client	43.913	52.7293	39.8993	33.39584	37.81973	41.55143
rocketmq-common	36.82744	42.84849	36.15545	35.80384	36.82071	37.69119
rocketmq-container	1.467065	1.467065	1.467065	1.485165	1.485165	1.474305
rocketmq-controller	17.50383	17.50383	17.50383	14.2414	14.2414	16.19886
rocketmq-example	2.572222	2.572222	2.572222	2.572222	2.572222	2.572222
rocketmq-filter	5.428982	5.428982	5.428982	5.428982	5.428982	5.428982
rocketmq-namesrv	13.80014	21.14752	19.13348	19.32422	19.33348	18.54777
rocketmq-openmessaging	8.74371	8.74371	8.74371	8.74371	8.74371	8.74371
rocketmq-proxy	81.00658	81.01599	76.39734	72.6178	74.7752	77.16258
rocketmq-remoting	90.8207	92.68279	87.51565	87.50117	88.09414	89.32289
rocketmq-srvutil	2.833333	2.833333	2.833333	2.833333	2.833333	2.833333
rocketmq-store	67.38242	67.43238	66.47285	60.499	61.47072	64.65147
rocketmq-test	48.38729	48.38729	48.38729	47.02056	47.28729	47.89394
rocketmq-tieredstore	20.13927	19.9701	29.47676	25.0251	26.2242	24.16708
rocketmq-tools	25.91216	26.89988	25.90942	25.96718	25.91279	26.12029

Fig. 12. TCC metric values

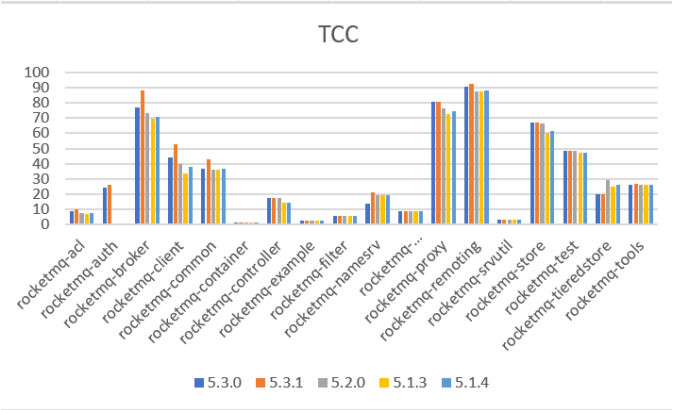


Fig. 13. TCC graph

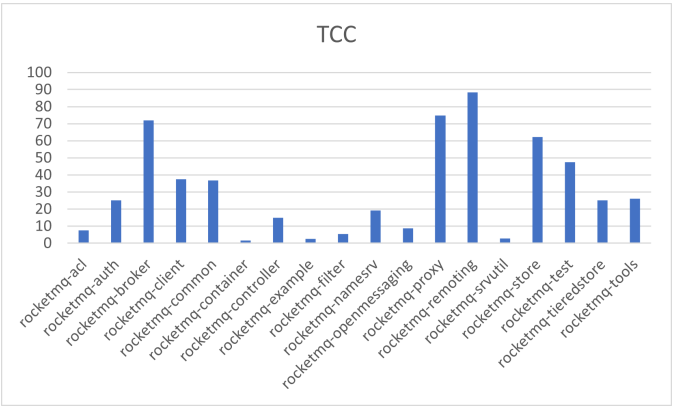


Fig. 14. TCC mean for each module

These two metrics combined demonstrate that RocketMQ has a mix of design quality across its modules. Some modules are well-structured and simple to manage, whereas others are more complex and less coherent. Specifically, ’rocketmq-store’ and ’rocketmq-common’ stand out as modules that

could benefit from code reworking to increase cohesiveness. Large, complex classes could be separated into smaller, more specialized ones to increase clarity and maintainability. Meanwhile, modules such as 'rocketmq-remoting' and 'rocketmq-broker', which have both a high LCOM and a high TCC, should be studied more extensively to determine whether the high complexity is warranted or if it conceals design errors.

In conclusion, this investigation demonstrates that cohesion is inconsistent throughout RocketMQ. Some modules are well-organized and cohesive, whereas others are vast and may cause problems. Developers and maintainers should prioritize the high-LCOM, low-TCC areas for improvement. This will make the codebase more understandable, extensible, and testable in the long run. Using cohesion metrics like LCOM and TCC can help uncover such issues early on and inform decisions about software restructuring and design enhancements.

Q2. How coupled is a RocketMQ module with other modules?

To determine how closely the RocketMQ modules are linked to other portions of the system, we used two typical software metrics: CBO (Coupling Between Object Classes) and RFC (Response For a Class). These two measurements demonstrate how dependent a module is on others. A high CBO score indicates that a module communicates with a large number of other classes, making it more unstable and difficult to update. Similarly, a high RFC value indicates that a module has numerous methods that can be invoked, implying that the class is vast, complex, and possibly attempting to do too much at once.

CBO Analysis: Looking at the CBO data, we discovered that some modules are quite strongly linked to others. For example, the 'rocketmq-proxy' module has the highest average CBO value, around 2594, followed by 'rocketmq-broker' (2552.6) and 'rocketmq-client' (2520). These modules are clearly vital to RocketMQ and are likely to coordinate numerous tasks, but their close coupling implies that changes in other sections of the system might readily affect them. This increases the danger of maintenance. On the other hand, certain modules, such as 'rocketmq-srvutil' (CBO = 23), 'rocketmq-container' (about 155), and 'rocketmq-openmessaging' (approximately 228), are significantly more self-contained. This is a good thing because it means that modules are less reliant on other elements of the system, making them easier to update or reuse.

CBO						
module	5.3.0	5.3.1	5.2.0	5.1.3	5.1.4	Mean
rocketmq-acl	250	258	245	233	245	246.2
rocketmq-auth	504	534				519
rocketmq-broker	2591	2846	2476	2425	2425	2552.6
rocketmq-client	2542	2768	2470	2364	2456	2520
rocketmq-common	569	632	538	464	527	546
rocketmq-container	157	157	155	152	154	155
rocketmq-controller	541	541	541	370	373	473.2
rocketmq-example	360	385	359	357	359	364
rocketmq-filter	187	194	194	194	194	192.6
rocketmq-namesrv	419	375	369	370	368	380.2
rocketmq-openmessaging	224	224	231	231	231	228.2
rocketmq-proxy	2739	2732	2544	2433	2523	2594.2
rocketmq-remoting	1464	1478	1275	1149	1184	1310
rocketmq-srvutil	23	23	23	23	23	23
rocketmq-store	1872	1832	1890	1635	1637	1773.2
rocketmq-test	1643	1647	1542	1605	1623	1612
rocketmq-tieredstore	580	580	606	485	539	558
rocketmq-tools	1764	1799	1603	1598	1616	1676

Fig. 15. CBO metric values

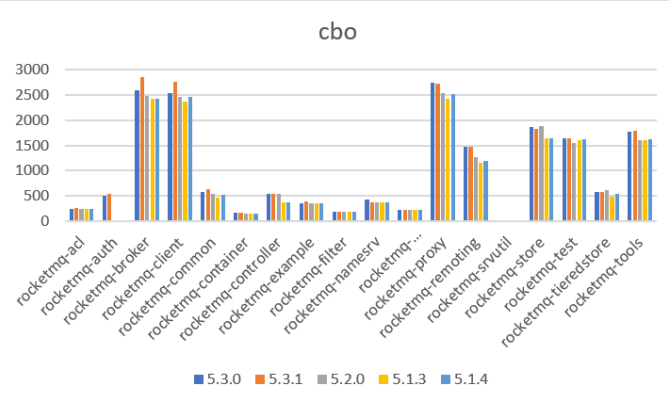


Fig. 16. CBO graph

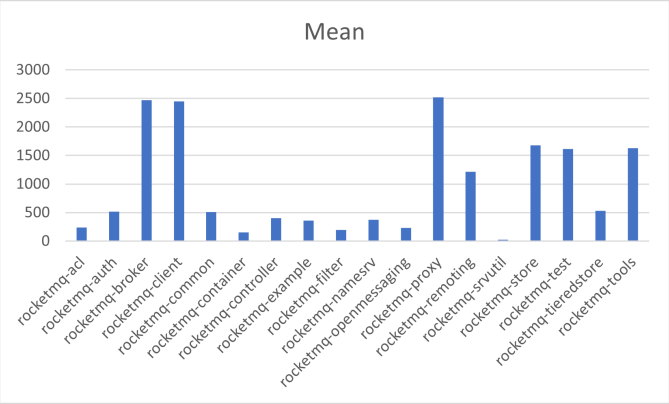


Fig. 17. CBO mean for each module

RFC Analysis: The RFC values support this pattern. We can see that 'rocketmq-broker', with an average RFC of roughly 8406, and 'rocketmq-client' (about 6289), are both quite sophisticated. This means they have a huge number of method calls, which increases the number of different execution routes and behaviors. The more methods a module needs to reply to, the more difficult it is to grasp what the

module is doing, and developers must exercise caution when updating the code. Other modules with high RFC values are 'rocketmq-store' (7210) and 'rocketmq-proxy' (5166). These high numbers indicate that these modules are not just closely linked, but also have several internal responsibilities.

RFC

module	5.3.0	5.3.1	5.2.0	5.1.3	5.1.4	Mean
rocketmq-acl	870	883	863	840	864	864
rocketmq-auth	979	1040				1009.5
rocketmq-broker	8527	9356	8212	7847	8090	8406.4
rocketmq-client	6360	7053	6102	5846	6083	6288.8
rocketmq-common	2051	2209	1997	1787	1997	2008.2
rocketmq-container	438	438	428	405	418	425.4
rocketmq-controller	1252	1252	1253	874	875	1101.2
rocketmq-example	991	1064	993	990	989	1005.4
rocketmq-filter	477	478	475	475	475	476
rocketmq-namesrv	1069	1022	984	963	974	1002.4
rocketmq-openmessaging	437	437	Plot Area	443	443	440.6
rocketmq-proxy	5285	5273	5202	4875	5199	5166.8
rocketmq-remoting	2904	2920	2860	2561	2680	2785
rocketmq-srvutil	93	93	93	91	93	92.6
rocketmq-store	7547	7527	7532	6713	6732	7210.2
rocketmq-test	3814	3819	3790	3722	3797	3788.4
rocketmq-tieredstore	2047	2069	2218	1723	1889	1989.2
rocketmq-tools	4131	4172	3797	3768	3819	3937.4

Fig. 18. RFC metric values

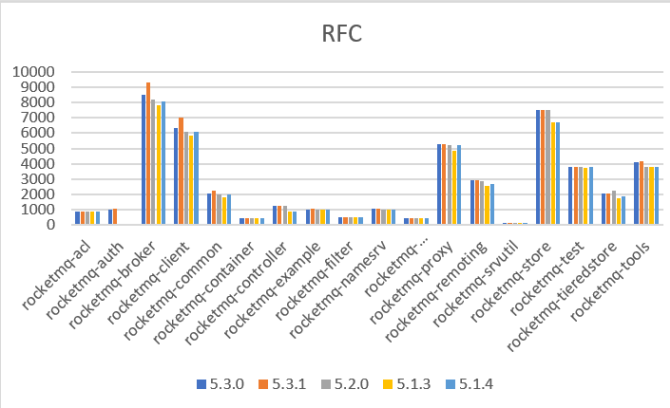


Fig. 19. RFC graph

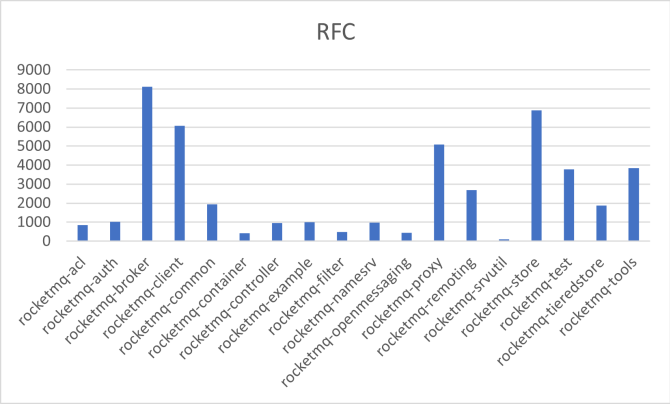


Fig. 20. RFC Mean for each module

Overall, this analysis reveals that some RocketMQ modules are fairly tightly connected and complex, whereas others are more lightweight and flexible. The essential modules—broker, client, proxy, and store—naturally have greater responsibilities, but their close coupling makes them more susceptible to change and more difficult to manage. Developers should use caution when altering these modules and evaluate whether some of their tasks should be divided into smaller, more focused classes. Meanwhile, the simpler modules demonstrate solid software design, with distinct boundaries and low reliance on the rest of the system. Improving the modularity of more complicated modules may result in a more reliable and manageable RocketMQ system in the long term.

Q3. How complex is the logic within a RocketMQ module? To determine the complexity of the logic within each RocketMQ module, we employed two significant object-oriented metrics: DIT (Depth of Inheritance Tree) and WMC (Weighted Methods per Class). These two measurements assist us in comprehending the various sorts of complexity. DIT indicates how deep a class is in the inheritance hierarchy—the number of layers it inherits from. A high DIT value indicates that the class may inherit behavior from multiple parent classes, which might make it difficult to comprehend or troubleshoot. WMC, on the other hand, displays the number of methods in a class as well as their level of complexity. A higher WMC usually indicates that the class is doing a lot of activities, which can be difficult to maintain.

DIT Analysis: The DIT data show that some modules have exceptionally deep inheritance trees, particularly 'rocketmq-remoting' with a mean DIT of 584.8 and 'rocketmq-proxy' with 447.2. These scores are relatively high, indicating that the classes in these modules may rely extensively on inheritance, which can increase complexity. Other modules with reasonably deep inheritance include rocketmq-store (DIT 413.4), rocketmq-client (395.2), and rocketmq-broker (323.2). Deep inheritance can make the flow of reasoning more difficult to follow, especially when attempting to grasp where a specific function or behavior comes from. Meanwhile, simpler modules such as 'rocketmq-container' and 'rocketmq-srvutil' have extremely shallow inheritance trees, with DIT values of 24 and 10, respectively. This demonstrates that the modules are structurally more straightforward.

DIT						
module	5.3.0	5.3.1	5.2.0	5.1.3	5.1.4	Mean
rocketmq-acl	44	46	43	42	43	43.6
rocketmq-auth	77	79				78
rocketmq-broker	334	345	328	294	315	323.2
rocketmq-client	388	396	412	374	409	395.8
rocketmq-common	277	302	265	254	263	272.2
rocketmq-container	24	24	24	24	24	24
rocketmq-controller	71	71	71	52	51	63.2
rocketmq-example	101	110	101	101	101	102.8
rocketmq-filter	86	86	86	86	86	86
rocketmq-namesrv	74	45	43	45	43	50
rocketmq-openmessaging	38	38	38	38	38	38
rocketmq-proxy	464	464	445	424	439	447.2
rocketmq-remoting	647	656	579	518	524	584.8
rocketmq-srvutil	10	10	10	10	10	10
rocketmq-store	428	429	427	391	392	413.4
rocketmq-test	310	310	310	301	307	307.6
rocketmq-tieredstore	94	94	100	82	86	91.2
rocketmq-tools	197	201	183	182	184	189.4

Fig. 21. DIT metric values

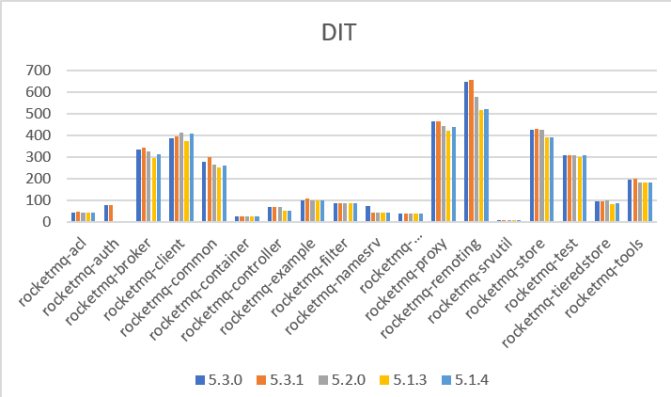


Fig. 22. DIT graph

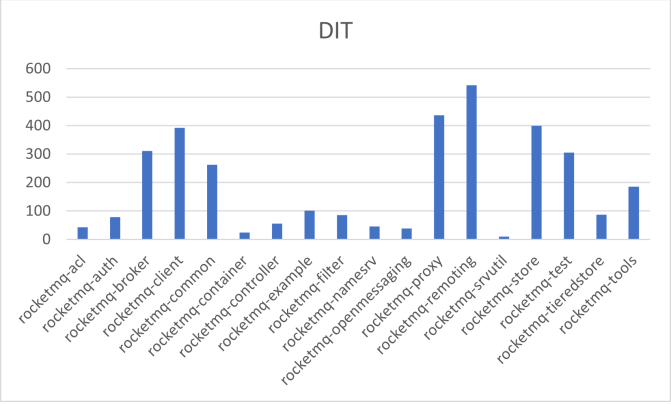


Fig. 23. DIT Mean for each module

WMC Analysis: When we look at the WMC numbers, which indicate how many and how difficult the methods in a class are, the picture becomes evident. The 'rocketmq-store' module has the greatest WMC (6644.8), followed by 'rocketmq-broker' (5606.2), 'rocketmq-client' (5299.8), and 'rocketmq-remoting' (4150.8). These high WMC values indicate that the classes in these modules have a big number of methods or particularly complex ones. This implies that the logic in these modules is deep, with several responsibilities handled within single classes, making testing, debugging, and adding new features more complex.

WMC						
module	5.3.0	5.3.1	5.2.0	5.1.3	5.1.4	Mean
rocketmq-acl	794	808	788	773	788	790.2
rocketmq-auth	961	974				967.5
rocketmq-broker	5708	6082	5547	5254	5440	5606.2
rocketmq-client	5462	5716	5248	4898	5175	5299.8
rocketmq-common	2830	2949	2776	2528	2718	2760.2
rocketmq-container	313	313	310	301	301	307.6
rocketmq-controller	918	918	918	658	658	814
rocketmq-example	508	526	508	508	508	511.6
rocketmq-filter	1521	1521	1521	1521	1521	1521
rocketmq-namesrv	921	731	705	691	701	749.8
rocketmq-openmessaging	352	352	352	352	352	352
rocketmq-proxy	3028	3037	2932	2749	2902	2929.6
rocketmq-remoting	4291	4307	4164	3975	4017	4150.8
rocketmq-srvutil	60	60	59	59	59	59.4
rocketmq-store	6922	6965	6898	6206	6233	6644.8
rocketmq-test	2023	2024	2023	1988	2017	2015
rocketmq-tieredstore	1250	1273	1529	1313	1377	1348.4
rocketmq-tools	2389	2463	2200	2183	2208	2288.6

Fig. 24. WMC metric values

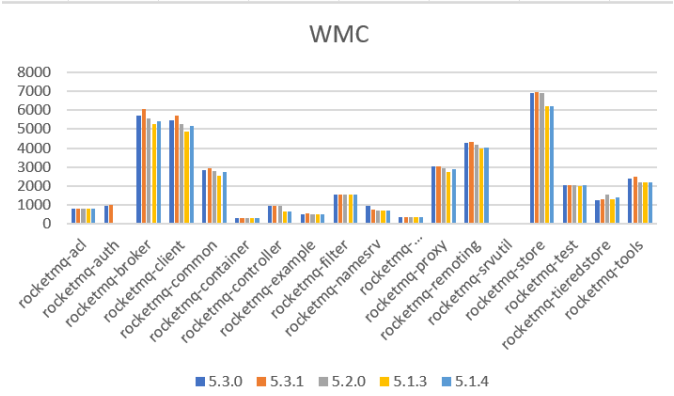


Fig. 25. WMC graph

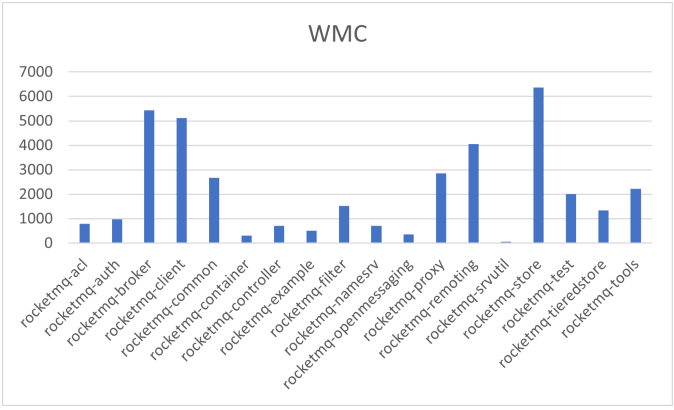


Fig. 26. WMC Mean for each module

Modules with lower WMC values, such as 'rocketmq-srvutil' (59.4), 'rocketmq-openmessaging' (352), and 'rocketmq-container' (307.6), handle simpler jobs with fewer methods or less sophisticated logic. These modules are more manageable and easier for developers to grasp. When we combine the DIT and WMC results, we can observe that the most difficult logic is centered in the key functional modules, such as 'rocketmq-broker', 'rocketmq-client', 'rocketmq-remoting', and 'rocketmq-store'. These modules feature both deep inheritance and high method complexity, making them more difficult to maintain and error-prone if not properly handled. In contrast, supporting or utility modules such assrvutil, container, and openmessaging are lightweight and simpler, resulting in a clearer structure and a lesser chance of complexity-related difficulties.

Finally, RocketMQ contains both basic and complicated modules. While the complicated modules are most likely vital to the system's basic functioning, they should be monitored and maybe refactored to eliminate deep inheritance and disperse responsibilities across smaller, more focused classes. Keeping logic simple and modular can help increase maintainability, minimize problems, and make development more efficient in the long run.

Q4.How understandable is a RocketMQ module's code?

To determine how easy it is for developers to read and understand RocketMQ code, we examined two crucial indicators: comment density and average method length. Comment density indicates how many comments exist in the code relative to its overall size. In general, higher comment density indicates that the code is well-documented, making it easier for others to comprehend what it is doing. Meanwhile, the average method length indicates how long the procedures are. Shorter methods are generally easier to read, test, and reuse. Long methods can make the code more difficult to comprehend since they frequently attempt to perform too much at once.

Comment Density Analysis: Looking at the comment density figures, we can see that most modules are within a reasonable range. Modules such as 'rocketmq-srvutil' (23.86), 'rocketmq-remoting' (23.576), 'rocketmq-common' (22.986), and 'rocketmq-example' (22.946) have high comment density, indicating that they are well-documented. This is a good sign, indicating that developers took the effort to describe what the code does. These modules are probably easier for new contributors to comprehend and maintain. On the other hand, modules such as 'rocketmq-auth' (6.31) and 'rocketmq-broker' (10.676) have very low comment density, indicating a lack of documentation. This can make it difficult for developers to understand the intent of the code, particularly in complex regions.

Comment Density						
module	5.3.1	5.3.0	5.2.0	5.1.4	5.1.3	Mean
rocketmq-acl	12.58	12.3	12.11	12.12	12.21	12.264
rocketmq-auth	15.72	15.84	0	0	0	6.312
rocketmq-broker	10.23	10.69	10.82	10.82	10.82	10.676
rocketmq-client	14.69	15.4	16.14	16.09	16.3	15.724
rocketmq-common	23.24	23.14	22.63	22.58	23.34	22.986
rocketmq-container	16.65	16.65	17.08	16.99	16.97	16.868
rocketmq-controller	17.87	17.87	17.87	19.33	19.31	18.45
rocketmq-example	23.15	22.89	22.89	22.91	22.89	22.946
rocketmq-filter	20.47	20.47	20.47	20.47	20.47	20.47
rocketmq-namesrv	12.03	12.06	12.06	12.11	12.14	12.08
rocketmq-openmessaging	17.68	17.68	17.68	17.68	17.68	17.68
rocketmq-proxy	14.19	14.23	14.24	14.19	14.29	14.228
rocketmq-remoting	23.26	23.19	23.46	23.9	24.07	23.576
rocketmq-srvutil	23.85	23.85	23.85	23.85	23.9	23.86
rocketmq-store	16.21	15.26	15.27	12.08	15.02	14.768
rocketmq-test	14.95	14.96	14.98	14.96	14.98	14.966
rocketmq-tieredstore	15.99	16.07	16.54	16.49	15.88	16.194
rocketmq-tools	14.77	14.87	14.89	14.92	14.91	14.872

Fig. 27. Comment density metric values

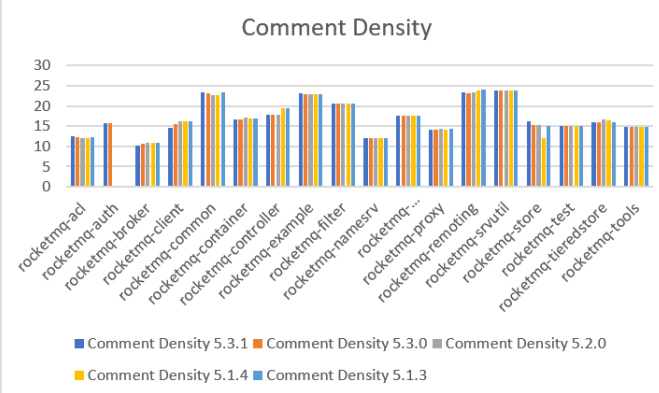


Fig. 28. Comment density graph

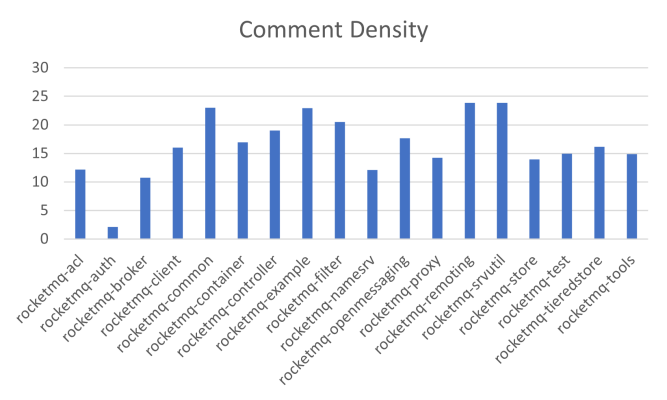


Fig. 29. Comment density Mean for each module

Average method length Analysis: The average method length varies significantly amongst modules. Some modules, including 'rocketmq-example' (111.88), 'rocketmq-test' (101.66), and 'rocketmq-srvutil' (112.06), have very long average methods, which can reduce code readability. These lengthy methods may be doing too much at once, making the reasoning harder to comprehend and the code more

complex to test or alter. In comparison, modules like 'rocketmq-controller' (12.76), 'rocketmq-common' (11.20), and 'rocketmq-auth' (11.43) contain much shorter methods, implying that their code is split down into smaller, more digestible chunks—something that is typically regarded as excellent programming practice.

Average method length						
module	5.3.0	5.3.1	5.2.0	5.1.3	5.1.4	Mean
rocketmq-acl	25.77714	26.23429	26.73054	72.11667	73.18033	44.80779
rocketmq-auth	11.3013	11.55844				11.42987
rocketmq-broker	19.3151	2.841635	19.1472	33.50435	33.2509	21.61183
rocketmq-client	12.92467	13.66606	12.39271	22.21716	22.36953	16.71402
rocketmq-common	9.434251	9.980138	8.71563	14	13.85932	11.19787
rocketmq-container	15.21698	15.21698	14.9717	23.1194	23.1194	18.32889
rocketmq-controller	8.638686	8.638686	8.638686	18.98343	18.92818	12.76553
rocketmq-example	63.24138	66.37931	63.24138	183.3	183.25	111.8824
rocketmq-filter	18.5233	18.5233	18.51971	26.49744	26.49744	21.71224
rocketmq-namesrv	17.63851	14.08446	13.23311	30.656	31.35484	21.39338
rocketmq-openmessaging	11.00641	11.00641	11.00641	16.35238	16.35238	13.1448
rocketmq-proxy	13.401	13.36424	13.4673	24.06046	23.97527	17.65365
rocketmq-remoting	9.625256	9.605999	9.779743	21.75949	22.05689	14.56548
rocketmq-srvutil	30.54545	30.54545	30.54545	111.6667	112	63.06061
rocketmq-store	14.39369	14.34856	14.34183	19.53991	19.4507	16.41494
rocketmq-test	62.68817	62.69892	62.68817	147.5714	150.6104	97.25142
rocketmq-tieredstore	13.99381	14.05521	17.32751	27.11024	26.95131	19.88762
rocketmq-tools	17.11385	16.85488	16.67587	61.42703	62.54348	34.92302

Fig. 30. Average method length values

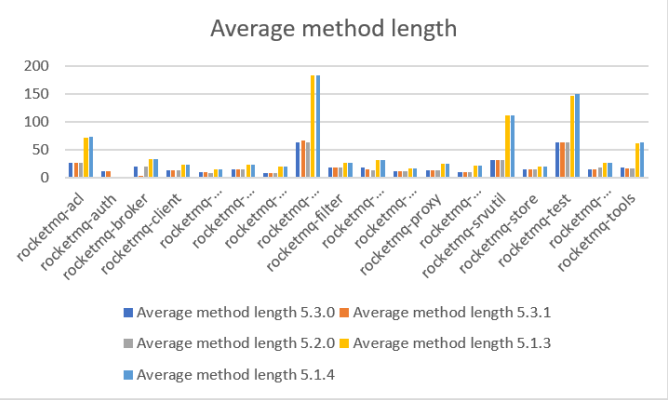


Fig. 31. Average method length graph

Combining these two criteria yields a more complete picture of module understandability. Modules like 'rocketmq-common' and 'rocketmq-remoting' perform well on both fronts: they feature high comment density and relatively short methods, making them easy to understand and maintain. On the other hand, 'rocketmq-example' and 'rocketmq-test' contain long methods, despite being well-documented. This shows that, while comments are useful, overall readability might be enhanced by breaking down large procedures into smaller ones.

Overall, the findings indicate that code understandability in RocketMQ varies greatly between modules. Some are quite well-written and documented, while others may benefit from additional comments and simpler method architectures.

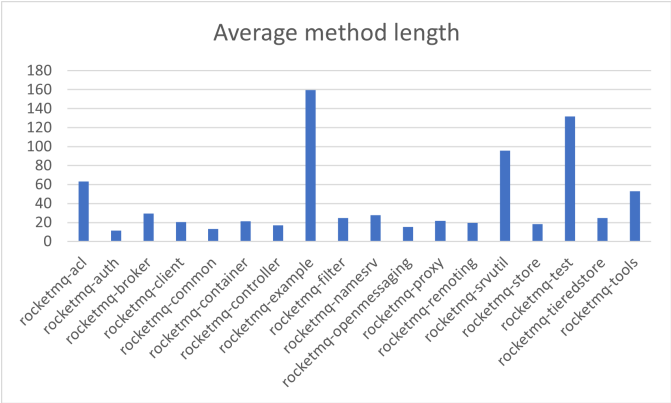


Fig. 32. Average method length Mean for each module

Improving understandability—particularly in key or often updated modules—can help decrease problems, accelerate new developer onboarding, and improve long-term maintenance efficiency. We advocate reworking large methods into smaller ones and adding comments to under-documented modules such as 'rocketmq-auth' and 'rocketmq-broker'.

Q5. What is the size of a RocketMQ module in terms of code and structure? We examined these variables to determine the size of each RocketMQ module, both in terms of code content and organization: Lines of code (LOC), Number of classes (NOC) and Number of methods (NOM). LOC displays how much code is generated in a module, NOC shows how many classes are utilized to structure that code, and NOM indicates how many methods (or actions) are implemented across those classes. Together, these allow us to evaluate modules based on size and complexity, determining which are more complex than others.

LOC Analysis: When we evaluated the LOC, we noticed that several modules had expanded significantly across versions. For example, 'rocketmq-store' had around 32,152 lines of code in version 5.1.4, which came to 35,814 in version 5.3.1. 'rocketmq-client' and 'rocketmq-broker' also had continuous growth, crossing 30,000 LOC in recent versions. This demonstrates that these modules are significant and potentially critical to RocketMQ's operation. In contrast, modules such as 'rocketmq-container', 'rocketmq-example', and 'rocketmq-srvutil' were small and stable under 4,000 lines across all versions, showing that they might be focused on specialized, simple tasks.

LOC					
module	5.3.0	5.3.1	5.2.0	5.1.3	5.1.4
rocketmq-acl	4511	4591	4464	4327	4464
rocketmq-auth	4315	4450			
rocketmq-broker	29301	32031	28357	26971	27831
rocketmq-client	28137	30120	26855	24861	26575
rocketmq-common	12340	13064	12045	10962	11822
rocketmq-container	1613	1613	1587	1549	1549
rocketmq-controller	4734	4734	4734	3436	3426
rocketmq-example	3668	3850	3668	3666	3665
rocketmq-filter	5168	5168	5167	5167	5167
rocketmq-namesrv	5221	4169	3917	3832	3888
rocketmq-openmessaging	1717	1717	1717	1717	1717
rocketmq-proxy	16108	16144	15649	14725	15512
rocketmq-remoting	18827	18895	18249	17190	17447
rocketmq-srvutil	336	336	336	335	336
rocketmq-store	35610	35814	35453	32065	32152
rocketmq-test	11660	11662	11660	11363	11597
rocketmq-tieredstore	6787	6873	7936	6886	7196
rocketmq-tools	12476	12776	11473	11364	11508

Fig. 33. LOC metric values

NOC					
module	5.3.0	5.3.1	5.2.0	5.1.3	5.1.4
rocketmq-acl	27	27	27	28	28
rocketmq-auth	61	61	-	-	-
rocketmq-broker	152	153	151	142	147
rocketmq-client	149	149	149	141	148
rocketmq-common	196	196	196	185	193
rocketmq-container	12	12	12	12	12
rocketmq-controller	68	68	68	52	52
rocketmq-example	65	69	65	65	65
rocketmq-filter	33	33	33	33	33
rocketmq-namesrv	40	40	40	40	40
rocketmq-openmessaging	15	15	15	15	15
rocketmq-proxy	143	143	137	133	137
rocketmq-remoting	266	269	248	245	247
rocketmq-srvutil	4	4	4	4	4
rocketmq-store	169	170	169	151	152
rocketmq-test	35	35	35	35	35
rocketmq-tieredstore	36	36	33	30	31
rocketmq-tools	71	74	70	68	69

Fig. 35. NOC metric values

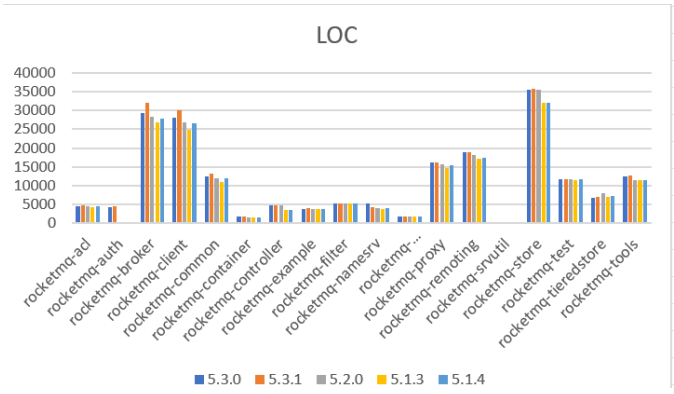


Fig. 34. LOC graph

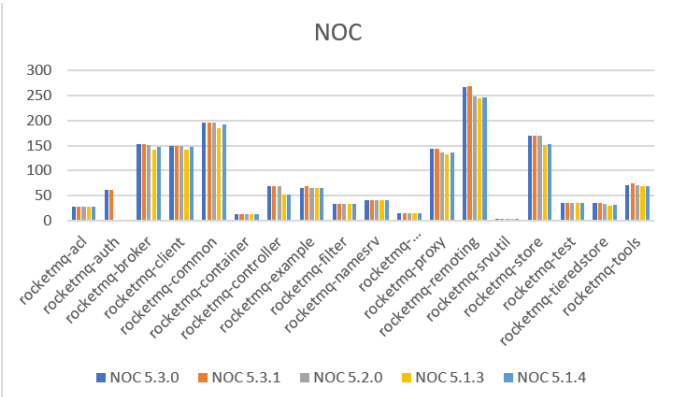


Fig. 36. NOC graph

NOC Analysis: In terms of order, we found that the number of classes (NOC) maintains an ongoing pattern. Modules such as 'rocketmq-remoting' and 'rocketmq-store' have the highest class counts, with 'rocketmq-remoting' having 269 classes in version 5.3.1. Other huge modules, such as 'rocketmq-proxy' and 'rocketmq-common', had more than 100 classes, demonstrating how these modules are divided into multiple components. Smaller modules, such as 'rocketmq-srvutil' and 'rocketmq-openmessaging', had fewer than 20 classes and were hence easier to manage and understand.

NOM Analysis: We also compared the number of methods (NOM) and observed a strong correlation between the number of methods and the size of the module. For example, 'rocketmq-store' contained 2,496 methods in version 5.3.1, up from 1653 in version 5.1.4. Similarly, 'rocketmq-client' and 'rocketmq-remoting' reached 2,000 methods. These figures indicate that these modules have a high level of functionality and perform a lot of work inside. Smaller modules, such as 'rocketmq-example' and 'rocketmq-srvutil', have fewer than 12 methods, suggesting fewer responsibilities.

NOM						
module	5.3.0	5.3.1	5.2.0	5.1.3	5.1.4	Mean
rocketmq-acl	175	175	167	60	61	127.6
rocketmq-auth	385	385	-	-	-	385
rocketmq-broker	1517	1566	1481	805	837	1241.2
rocketmq-client	2177	2204	2167	1119	1188	1771
rocketmq-common	1308	1309	1382	783	853	1127
rocketmq-container	106	106	106	67	67	90.4
rocketmq-controller	548	548	548	181	181	401.2
rocketmq-example	58	58	58	20	20	42.8
rocketmq-filter	279	279	279	195	195	245.4
rocketmq-namesrv	296	296	296	125	124	227.4
rocketmq-openmessaging	156	156	156	105	105	135.6
rocketmq-proxy	1202	1208	1162	612	647	966.2
rocketmq-remoting	1956	1967	1866	790	791	1474
rocketmq-srvutil	11	11	11	3	3	7.8
rocketmq-store	2474	2496	2472	1641	1653	2147.2
rocketmq-test	186	186	186	77	77	142.4
rocketmq-tieredstore	485	489	458	254	267	390.6
rocketmq-tools	729	758	688	185	184	508.8

Fig. 37. NOM metric values

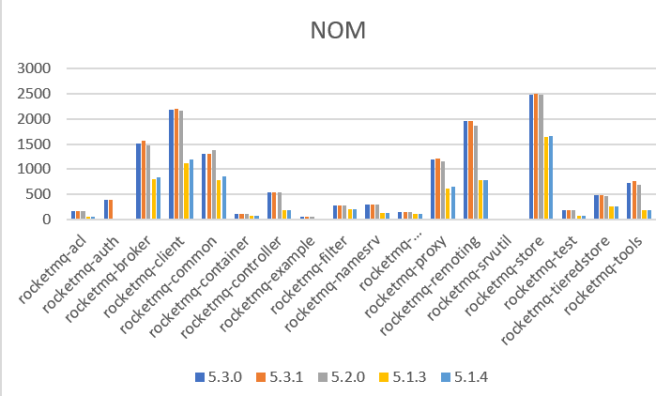


Fig. 38. NOM graph

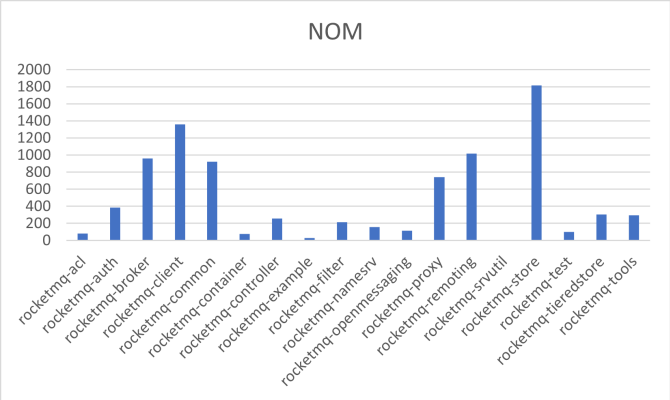


Fig. 39. NOM Mean for each module

The size of a RocketMQ module, determined by the number of methods (NOM), varies greatly across the system. Some modules, such as rocketmq-store, rocketmq-client, and rocketmq-remoting, have over a thousand methods each, indicating that they are huge, complex, and handle a wide range of tasks. These modules are likely to comprise the system's core, although their size may make them more difficult to manage or understand. Smaller modules, such as rocketmq-srvutil, rocketmq-example, and rocketmq-container, offer fewer methods and hence provide simpler, more focused functionality. This contrast demonstrates that, while certain aspects of RocketMQ are self-contained, others could benefit from being broken down into smaller, more manageable chunks to increase clarity and maintainability.

In general, our research indicates that RocketMQ has both very large, core modules (like 'rocketmq-store', 'rocketmq-client', and 'rocketmq-remoting') and smaller, more specific ones (like 'rocketmq-srvutil' and 'rocketmq-example'). The large modules are continuously increasing in both code and structure over time, indicating that they contain more features and complexity. Meanwhile, the smaller modules are still reliable, clean, and easy to use. This size difference is useful to consider when planning RocketMQ maintenance, refactoring, or testing projects.

D. Aggregation of Results:

After reviewing the results of our five questions, we can conclude that RocketMQ contains a mix of easy and difficult-to-maintain modules. Some parts of the system are tidy, straightforward, and well-organized. Others are very large, complex, and interconnected with too many other parts, making them difficult to understand and care for.

Rocketmq-store, rocketmq-client, and rocketmq-remoting are three modules that do a lot of work. They have numerous methods, lengthy and complex logic, and are intricately linked to other modules. These are likely the most important components of RocketMQ, but they are also the most difficult to manage and update. Changes in one location can have far-reaching consequences.
Changes in one location can have far-reaching consequences. These modules also have long methods with fewer comments, making the code more difficult to read and work with.

Smaller modules, such as rocketmq-srvutil, rocketmq-example, and rocketmq-container, are more understandable and maintainable. They use fewer methods, write shorter, more focused code, and provide better comments. This type of design is beneficial because it makes the code clearer and easier to work with, particularly for new developers.

Overall, RocketMQ is expanding and adding new features, but some parts are becoming too large and complicated. To make the project easier to maintain in the future, developers should focus on improving large, complex modules—by adding more comments, breaking long methods down into smaller ones, and reducing their reliance on other modules. This will improve the system's reliability, make it easier to test, and help new team members understand it.

V. DISCUSSIONS

A. Analysis of Results:

After analyzing all of the data, we discovered some interesting relationships between the size of RocketMQ modules and how easy or difficult they are to work with. Overall, the larger the module, the more difficult it is. Modules such as rocketmq-store, rocketmq-client, and rocketmq-broker, for example, contain several methods and highly complex logic. These huge modules are more difficult to comprehend and fix because they perform multiple tasks at once and frequently depend on numerous other components of the system.

We also discovered that larger modules are more difficult to read and understand. They frequently have long methods and few comments, making it difficult for developers to understand what the code is doing. Even experienced coders can become confused in the absence of clear explanations. However, smaller modules, such as rocketmq-srvutil and rocketmq-container, are typically easier to understand. They use less approaches, more remarks, and concentrate on doing one task well. This makes them easier to maintain and less subject to cause issues.

Another finding we made is that some of the modules that change frequently are also the ones that are poorly structured. For example, rocketmq-broker and rocketmq-client are frequently updated, but they are also complex and difficult to understand. This is an importance because code that is difficult to comprehend and frequently changes is more likely to fail or cause errors.
So, in simple terms, we discovered that big modules are usually more complicated and hard to understand, whereas smaller modules are simpler and cleaner to work with. To improve RocketMQ's maintainability, consider dividing down larger and more complex modules into smaller parts, adding more comments, and reducing long methods.

B. Reflections

What did you learn from this project? This project taught us a lot about systematically evaluating and improving software maintainability with real-world tools and metrics. One of the most important takeaways was learning how to use various metric collection tools such as CK Master, MetricsReloaded, SonarQube, and CodeMR. Each tool had its own set of strengths—CK Master was ideal for in-depth class-level OO metrics, MetricsReloaded provided metrics such as CLOC, SonarQube provided useful visual dashboards and quick overviews, and CodeMR allowed us to delve deeper into structure and design details. Working with these tools made the concept of software quality seem much more tangible.
Furthermore, we learned how to analyze these metrics using descriptive statistics, visualizations, and correlation analysis. It was eye-opening to see how size, complexity, cohesion, and coupling interact in an open-source system like RocketMQ. We also gained confidence in raw data processing, Python-based metric aggregation, and using GQM to structure our study. Overall, the project contributed to the understanding of theoretical software metrics and their practical application.

Which of the steps you find more challenging to implement? The most difficult step in the GQM framework for us was undoubtedly developing meaningful and answerable questions (Step 2). While setting a high-level goal was relatively simple, breaking it down into clear, measurable questions required a lot of thought and iteration. We needed to make sure that each question was closely related to the maintainability goal while also being specific enough to guide the metric selection process. Finding the right balance between being too broad and too detailed was difficult, but it greatly helped us clarify the direction of our analysis.

What are your views about the two workshops? Did they help you apply the GQM framework in some way? Both workshops proved highly helpful at various stages of our project. The first workshop was extremely helpful in developing our GQM tree, as it provided guidance on how to set meaningful goals, write effective questions, and select appropriate metrics. It also allowed us to gather feedback and refine our thinking before beginning the actual analysis.

The second workshop was especially useful as we moved into the data analysis phase. It taught us how to apply the GQM tree to real metric data, identify patterns, and draw conclusions using visual and statistical techniques. Together, these workshops provided a clear path from theory to practice.

Additionally, the progress meetings we had through the course played an important role. They kept us on track, advised us to improve our performance at each stage, and gave us time to ask questions and reflect. Thanks to these inspection points, we not only got better results, but we also learned a lot about teamwork, continuous improvement, and how to structure an empirical software engineering study better.

REFERENCES

[1] "[ANN] MetricsReloaded 0.1 released," Apr. 2004. [Online]. Available: http://intellij-support.jetbrains.com/hc/en-us/community/posts/206132059–ANN-MetricsReloaded-0-1-released

[2] "CodeMR | Measure, visualise, and improve code quality | Better Code Better Quality!" Dec. 2020. [Online]. Available: https://www.codemr.co.uk/

[3] R. Harrison, S. Counsell, and R. Nithi, "Coupling metrics for object-oriented design," in *Proceedings Fifth International Software Metrics Symposium. Metrics (Cat. No.98TB100262)*, Nov. 1998, pp. 150–157. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/731240

[4] T. Honglei, S. Wei, and Z. Yanan, "The Research on Software Metrics and Software Complexity Metrics," in *2009 International Forum on Computer Science-Technology and Applications*, vol. 1, Dec. 2009, pp. 131–136. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/5385114

[5] A. Oram and G. Wilson, *Making Software: What Really Works, and Why We Believe It.* "O'Reilly Media, Inc.", Oct. 2010.

[6] M. Perepletchikov, C. Ryan, and K. Frampton, "Cohesion Metrics for Predicting Maintainability of Service-Oriented Software," in *Seventh International Conference on Quality Software (QSIC 2007)*, Oct. 2007, pp. 328–335, iSSN: 2332-662X. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/4385516

[7] S. Prykhodko, N. Prykhodko, and T. Smykodub, "A Joint Statistical Estimation of the RFC and CBO Metrics for Open-Source Applications Developed in Java," in *2022 IEEE 17th International Conference on Computer Sciences and Information Technologies (CSIT)*, Nov. 2022, pp. 442–445, iSSN: 2766-3639. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10000457

[8] H. Zuse and P. Bollmann, "Software metrics: using measurement theory to describe the properties and scales of static software complexity metrics," *SIGPLAN Not.*, vol. 24, no. 8, pp. 23–33, Aug. 1989. [Online]. Available: https://dl.acm.org/doi/10.1145/70470.70473